

```
        return long_name.title()
```

```
❸ my_new_car = Car('audi', 'a4', 2024)
   print(my_new_car.get_descriptive_name())
```

In the `Car` class, we define the `__init__()` method with the `self` parameter first ❶, just like we did with the `Dog` class. We also give it three other parameters: `make`, `model`, and `year`. The `__init__()` method takes in these parameters and assigns them to the attributes that will be associated with instances made from this class. When we make a new `Car` instance, we'll need to specify a `make`, `model`, and `year` for our instance.

We define a method called `get_descriptive_name()` ❷ that puts a car's `year`, `make`, and `model` into one string neatly describing the car. This will spare us from having to print each attribute's value individually. To work with the attribute values in this method, we use `self.make`, `self.model`, and `self.year`. Outside of the class, we make an instance from the `Car` class and assign it to the variable `my_new_car` ❸. Then we call `get_descriptive_name()` to show what kind of car we have:

```
2024 Audi A4
```

To make the class more interesting, let's add an attribute that changes over time. We'll add an attribute that stores the car's overall mileage.

Setting a Default Value for an Attribute

When an instance is created, attributes can be defined without being passed in as parameters. These attributes can be defined in the `__init__()` method, where they are assigned a default value.

Let's add an attribute called `odometer_reading` that always starts with a value of 0. We'll also add a method `read_odometer()` that helps us read each car's odometer:

```
class Car:
```

```
    def __init__(self, make, model, year):
        """Initialize attributes to describe a car."""
        self.make = make
        self.model = model
        self.year = year
❶    self.odometer_reading = 0
```

```
    def get_descriptive_name(self):
        --snip--
```

```
❷    def read_odometer(self):
        """Print a statement showing the car's mileage."""
        print(f"This car has {self.odometer_reading} miles on it.")
```

```
my_new_car = Car('audi', 'a4', 2024)
print(my_new_car.get_descriptive_name())
my_new_car.read_odometer()
```
